

reverse 정리

지정한 범위의 원소 순서를 제자리에서 반대로 뒤집는 C++ STL 알고리즘입니다.

사용처 (Where)

- vector나 배열의 순서를 뒤집을 때
- string의 문자 순서를 반대로 바꿀 때
- 전체가 아닌 부분 범위만 뒤집을 때

방법 (How to Use)

- 뒤집을 [first, last) 범위 지정
- reverse(first, last) 호출
- 원본 컨테이너가 직접 변경됨
- 반환값은 없음 (void)

기본 정보

- 필요 헤더: #include <algorithm>
- 기본 문법: reverse(v.begin(), v.end());
- string도 begin(), end()로 동일하게 사용
- 부분 범위: reverse(v.begin()+1, v.begin()+4);

왜 써야 할까? (Why)

- 직접 교환 반복문을 작성할 필요가 없음
- 추가 컨테이너 없이 제자리에서 처리
- 전체와 부분 범위에 모두 적용 가능

주요 함수

함수/문법	의미
reverse(first, last)	[first, last) 범위의 순서를 반전
reverse(v.begin(), v.end())	vector 전체 반전
reverse(s.begin(), s.end())	string 전체 반전
reverse(first+1, first+4)	인덱스 1, 2, 3 부분만 반전
reverse_copy(...)	원본은 두고 반전 결과를 복사

기본 코드 예시

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v{1, 2, 3, 4};
    reverse(v.begin(), v.end());
    for (int x : v) cout << x << ' ';
}
```

reverse 비교와 응용

직접 교환과 비교하고, 문자열과 부분 범위에 적용하는 방법을 확인합니다.

시간 복잡도

동작	복잡도
reverse(first, last)	$O(n)$
교환 횟수	약 $n / 2$ 회
추가 공간	$O(1)$
부분 범위 반전	$O(\text{범위 길이})$

STL vs 직접 구현 vs C 스타일

구분	STL reverse	직접 교환	새 역순 복사
표현력	의도가 명확	양끝 인덱스 관리	결과를 별도 보관
코드 길이	한 줄로 간결	반복과 swap 필요	복사 코드 필요
추가 공간	$O(1)$	$O(1)$	$O(n)$
추천 상황	원본을 바꾸어도 될 때	특수 규칙이 필요할 때	원본을 보존해야 할 때

응용: 문자열 뒤집기

```
#include <algorithm>
#include <iostream>
#include <string>
using namespace std;

int main() {
    string word = "coding";
    reverse(word.begin(), word.end());
    cout << word << '\n';
}
```

처음 배울 때 자주 하는 실수

- reverse()가 반전된 컨테이너를 반환한다고 생각함
- 두 번째 반복자 last가 범위에 포함된다고 생각함
- 부분 범위의 끝 위치에 +1을 빼먹음
- 원본을 보존해야 하는데 reverse()로 직접 변경함

비슷한 항목과 차이

- reverse_copy는 원본을 유지하며 반전된 결과를 다른 범위에 복사
- sort는 값의 크기를 기준으로 정렬하지만 reverse는 현재 순서만 반전
- ranges::reverse(C++20)는 컨테이너 범위를 더 간결하게 전달 가능
- string의 역방향 순회만 필요하면 rbegin(), rend()도 사용 가능

한 줄 정리: reverse는 '원본 범위의 순서를 그 자리에서 뒤집을 때' 선택하는 STL입니다.

사용법 숙지를 위한 기본 예제

기본 예제 1. vector 전체 뒤집기

begin()부터 end() 직전까지의 전체 순서를 반전합니다.

```
vector<int> v{1, 2, 3};
reverse(v.begin(), v.end());
for (int x : v) cout << x << ' ';
```

실행 결과: 3 2 1

기본 예제 2. 부분 범위만 뒤집기

인덱스 1부터 3까지만 뒤집으려면 끝에 +4를 전달합니다.

```
vector<int> v{1, 2, 3, 4, 5};
reverse(v.begin() + 1, v.begin() + 4);
for (int x : v) cout << x << ' ';
```

실행 결과: 1 4 3 2 5

reverse 현업 활용

경로 복원과 표시 순서 변환에서 수집된 역순 데이터를 올바른 순서로 바꾸는 활용을 살펴봅니다.

현업에서는 이렇게 사용합니다

사례 1. 경로 탐색 - 목적지에서 출발지 순서로 복원

주로 쓰이는 상황 BFS·다익스트라에서 부모 노드를 따라 목적지부터 경로를 모았을 때

실무 처리 흐름 목적지부터 parent 추적 -> 역순 경로 생성 -> reverse로 출발지 순서 변환

```
vector<int> path;
int cur = destination;

while (cur != -1) {
    path.push_back(cur);
    cur = parent[cur];
}

reverse(path.begin(), path.end());
// path: start -> ... -> destination
```

그래프 경로 복원에서 가장 대표적인 사용 패턴

사례 2. 화면 표시 - 오래된 기록부터 정렬해 보여주기

주로 쓰이는 상황 API가 최신 기록부터 반환했지만 화면은 시간 순서대로 표시해야 할 때

실무 처리 흐름 최신순 데이터 수신 -> 표시용 vector 복사 -> reverse 후 UI에 전달

```
vector<Log> display_logs = api_logs;

// api_logs: newest -> oldest
reverse(display_logs.begin(), display_logs.end());

// display_logs: oldest -> newest
render_timeline(display_logs);
```

원본 순서가 필요하면 복사본에 reverse 적용

실무 포인트

reverse는 정렬이 아니라 현재 순서의 반전입니다. 원본 보존 여부와 부분 범위의 끝 반복자를 먼저 확인합니다.